MedGraph

Agentic clinical document intelligence — a system designed to decline

MedGraph answers questions about public healthcare policy and drug-label documents — and refuses, with a stated reason, when its corpus cannot support an answer. It is built as a LangGraph state machine in which declining is a first-class path through the graph rather than an error case.

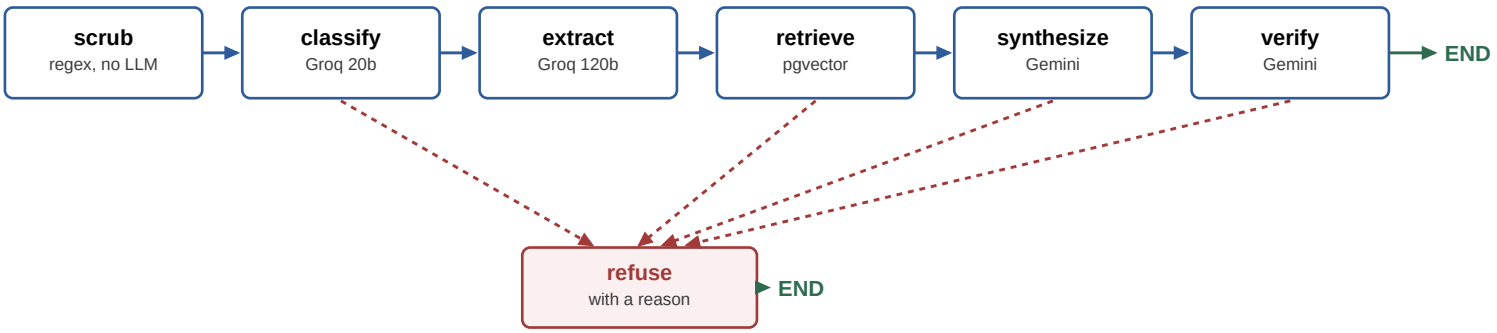
THE PROBLEM IT ADDRESSES

Most retrieval-augmented systems answer every question put to them. In healthcare that is the failure mode, not the feature: a confident wrong answer about coverage criteria or drug dosing is materially worse than no answer at all, because it is indistinguishable from a correct one and carries the same authority.

MedGraph inverts that default. Four properties are enforced **structurally** — in the type system and the graph topology — rather than requested in a prompt, because prompt instructions are advisory and types are not:

- **Grounded or refused.** The `Answer` model fails validation if it carries no citation and is not marked a refusal. An ungrounded answer cannot be constructed in the first place.
- **Citations cannot be invented.** The synthesis step returns *source ids*, never citation objects. Any id the retriever did not actually return is discarded; if none survive, the system refuses.
- **No identifiers reach a provider.** Redaction runs at the entry node, before the first model call — not as a post-processing step.
- **It is not a clinician.** Questions seeking an individual treatment decision are routed to refusal, not hedged with a disclaimer.

ARCHITECTURE



every red edge is a conditional path — declining is first-class

Seven nodes. State is checkpointed with MemorySaver, which is process-local: enough to resume a thread inside a running process, not enough to survive a crash or redeploy.

Node	Responsibility	Model
scrub	Redacts MRN, SSN, NPI, DOB, phone and email before anything reaches a provider	—
classify	Four-way intent, schema-constrained decoding	Groq gpt-oss-20b
extract	Typed clinical entities; codes recorded only when stated, never inferred	Groq gpt-oss-120b
retrieve	Exact cosine search over pgvector; returns nothing below a similarity floor	—
synthesize	Draft answer plus the ids of sources actually used	Gemini Flash-Lite
verify	Claim-to-citation entailment check over the draft	Gemini Flash-Lite
refuse	Terminal node carrying a reason — never an empty response	—

DESIGN DECISIONS WORTH EXPLAINING

Decision	Reasoning

Providers split across two vendors	Groq's free tier allows 200,000 tokens per model per day, and synthesis alone consumes ~3,800 of the ~6,200 tokens in a run. Moving the two token-heavy nodes to Gemini raises daily capacity from ~52 runs to ~222.
Constrained decoding, explicitly requested	The default structured-output method carries no schema guarantee. The provider layer requests JSON-schema mode by name, and **raises** if a node is routed to a model that would silently downgrade to best-effort JSON.
Optional fields carry no defaults	Strict mode rejects any property absent from the schema's required list, and a field with a default is emitted as optional. Optionality is therefore modelled as required-but-nullable.
Embeddings run locally	Not for cost — the whole corpus is only ~106,000 tokens, a rounding error on any free tier. The reasons are that the retrieval path gains no network dependency and no per-query round trip, embedding quota never contends with the model nodes' quota, and re-chunking is free and unmetered, which matters because chunk tuning is iterative.
No ANN index	At ~5,000 chunks an exact scan is single-digit milliseconds at 100% recall. An IVFFlat index at that size would scan roughly 1% of the corpus and present as broken embeddings.
Chunking against the real tokenizer	The embedding model truncates at 512 tokens *silently*, so an oversized chunk loses its tail with no error raised. A word ceiling alone proved insufficient: a 350-word limit derived from a ~1.3 tokens/word rule of thumb still produced chunks of 521 and 557 tokens on token-dense clinical text. The tokenizer is now authoritative; the maximum chunk is 448 tokens.
Token accounting per call, not per run	Free-tier budgets are enforced per model, and a single run spans three of them. One row per call makes the daily ceiling actually checkable.

MEASURED RESULTS

Retrieval is measured on the real corpus and needs no API key — embeddings run locally and the search is arithmetic — so these figures are reproducible offline.

Corpus	128 source documents → 339 chunks (~106k tokens; largest chunk 448 tokens)
recall@k	**1.0** — every evaluated case retrieves its expected document
precision@k	0.487
Query embedding	71 ms median on an idle two-core 2015 CPU; the same code measured 1,771 ms under load. On this hardware latency is dominated by contention, not the model.
Test suite	68 tests, passing offline with no API key or database

On precision. It is measured chunk-level against document-level expectations, so it is bounded by how many chunks a document has: retrieving six chunks when the expected document holds two caps precision at 0.33 even under perfect retrieval. Recall governs whether an answer can be grounded at all, which is why it leads. **And a caveat on recall 1.0:** the thirteen evaluated queries were written by the author and several restate a target document's title, so the figure means ‘no regression signal yet’ rather than ‘retrieval is solved’.

WHAT THE EVALUATION CAUGHT

The harness has already found four defects, two of them in itself — which is the usual pattern: the first thing an evaluation catches is its own assumptions.

- The grounding invariant did not work. A validator on the citations field referenced a flag declared after it, which the validator could never see, and a minimum-length rule made refusals impossible to construct at all.
- The advice guardrail refused *“What are anticoagulants prescribed for?”* — a bare trigger word caught legitimate drug-class questions. Now anchored to first-person phrasing.
- The chunker silently discarded every document whose paragraphs were all short, removing them from the corpus with no error.
- Three expected documents in the evaluation set did not exist; retrieval had been returning the correct document and being scored as a miss. Separately, reported query latency was roughly seventy times too high, because the timer wrapped a one-time model load and a Python loop the database performs in C.

CURRENT STATE AND HONEST LIMITATIONS

Implemented	Not yet

Graph topology with conditional refusal paths	Coverage-determination corpus is not ingested, so coverage questions currently refuse — and the evaluation set asserts that they should
Typed provider boundaries and constrained decoding	
Identifier redaction and the advice boundary	
Anti-fabrication citation guard	
Corpus ingestion, chunking and local embedding	The four model-backed nodes have not been run against live providers, so grounding and refusal accuracy are unmeasured end to end
Scored evaluation harness and retrieval baseline	
68 offline tests	Per-model daily token budgets are written and unit tested, but nothing calls them — no run has written a row to the runs table
	No real users; latency and failure behaviour under load are unknown

This is a portfolio system, not a compliant one. The corpus is public documents only. Redaction by pattern matching is a safety measure, not a compliance control, and the free provider tiers it runs on are not covered by any data-processing agreement. It must not be used with real patient information.

REPOSITORY MAP

Path	What it holds
src/medgraph/graph.py	The state machine: seven nodes, the four conditional edges, and the fabrication guard that drops source ids the retriever never returned
src/medgraph/schemas.py	Every typed boundary. The grounding invariant lives in Answer's model validator
src/medgraph/providers.py	Per-node model routing, explicit JSON-schema mode, and the guard that refuses a silent downgrade to best-effort JSON
src/medgraph/guardrails.py	Identifier redaction and the individual-advice boundary
src/medgraph/chunking.py	Paragraph-first chunking. Two tested invariants: nothing exceeds the embedding window, nothing is discarded
src/medgraph/embeddings.py	Local sentence-transformers behind LangChain's Embeddings interface
src/medgraph/budget.py	Per-model daily token ceilings, read from the runs table
scripts/ingest.py	fetch → chunk → embed → load, checkpointed and resumable
eval/	Golden set, scored harness, retrieval-only harness, published baseline
tests/	68 tests. Every graph path, including all five refusal paths, runs offline

RUNNING IT

Everything below the provider boundary runs with no API key. The test suite and the retrieval baseline are reproducible offline.

uv pip install -e ".[dev]"	Install. No key needed
pytest	68 tests, offline
python scripts/ingest.py --dry-run	Fetch and chunk only — no model, no database
python scripts/ingest.py --out data/vectors.jsonl	Embed the corpus locally (~50 min, resumable)
python eval/retrieval_eval.py	Retrieval baseline — no API key
python eval/run_eval.py	Full evaluation — needs free provider keys and a database

STACK

Python 3.12 • LangGraph • LangChain • Pydantic v2 • FastAPI • PostgreSQL with pgvector • Docker • sentence-transformers